Report on security vulnerabilities in Flask web applications and relational databases

Contents

Introduction

Security vulnerabilities are found in most software, especially within web applications and relational databases. Databases store high value data that adversaries want to get their hands on, and web applications can be one of the most direct routes to obtaining this, which is why it is imperative to ensure vulnerabilities are prevented rather than ignored.

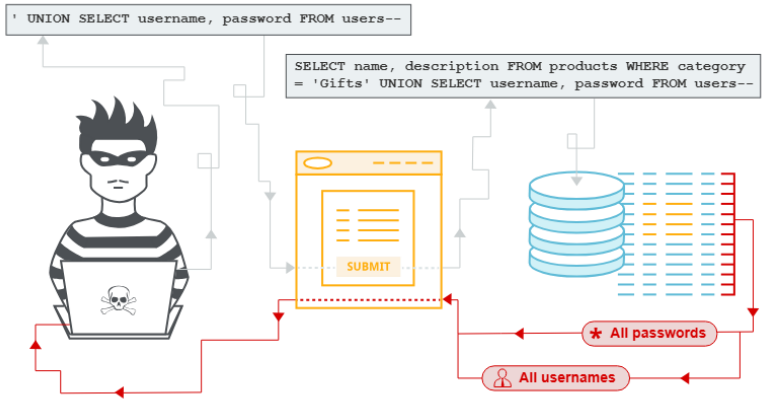
Not only is it good practice that vulnerabilities are stopped in the first instance for reasons like keeping the trust of customers, but it is also legally required within the confines of GDPR and Data Protection regulations.

Vulnerabilities

When vulnerabilities are identified and exploited, it will most of the time lead to a bad outcome. It depends entirely on the threat actor goals, which can vary in depth, from data exploitation to shutting down key IT systems. In the case of this report, it will focus more so on the exploitation of the database and the data stored within them and how they are exploited in practice.

SQL Injection

PortSwigger (no date) notes that SQL Injection can expose sensitive data including passwords, financial information, and personal details as further illustrated with this image:



In the OWASP Top 10, injection was defined as the top 5 in terms of the most critical security risks in web applications; it goes into explaining SQL and NoSQL, as well as select others, are the most exploited (The OWASP® Foundation, Inc., 2025.) At the same time MITRE (The MITRE Corporation., 2025) placed SQL Injection at the top two position. It is important to note that both OWASP and MITRE have placed these in their Top 5, which tells us that this is a critical exploit which must be addressed which can be done through simple configuration steps.

Prevention

An example of SQL Injection, as defined in the OWASP Top 10 A05 Injection, an attack could input malicious input on the 'id' field of a URL as **' OR '1'='1** which would be inserted into the SQL Query: "**SELECT * FROM accounts WHERE custID='" + request.getParameter("id") + "';"** (The OWASP® Foundation, Inc., 2025) This would show all accounts in a database, regardless if the custID matches or not and is your most basic kind of injection example. This could show credit card details, passwords, PII or anything else stored in plaintext. A similar injection attack was exploited on WordPress where they were "affected by a SQL injection vulnerability that allowed authenticated users to interfere with database queries" (Mascellino, 2026)

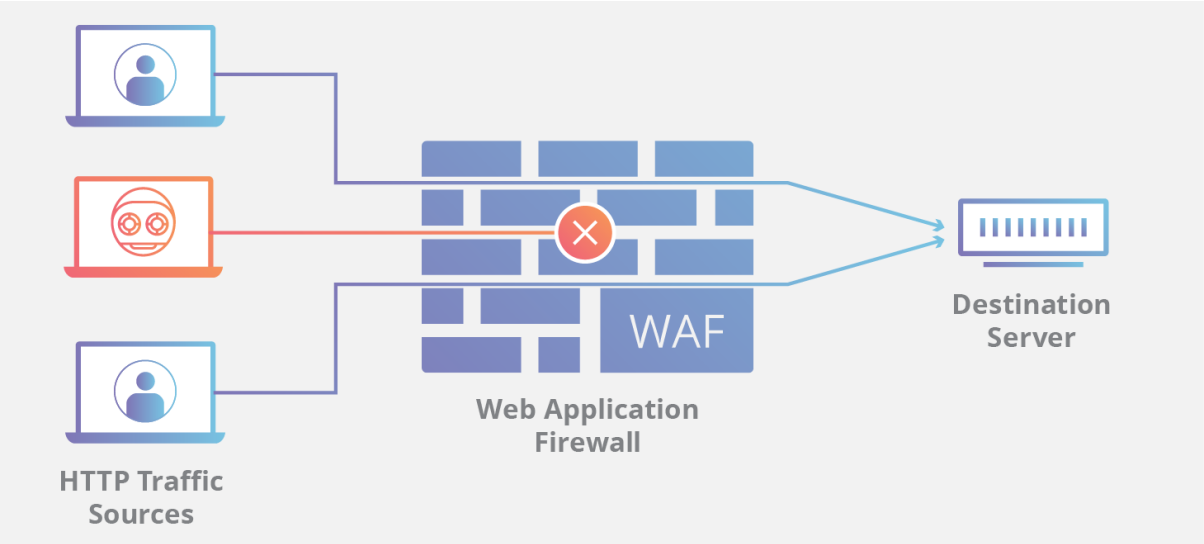
This kind of attack can be mitigated in several ways, but the main suggested defined by the OWASP Top 10 (2025) is to use a safe API with parameterised queries as “Even when parameterized, stored procedures can still introduce SQL injection”. This isn’t to say queries should not be parameterised, they should still be parameterised when available.

Cross-Site Scripting (XSS)

“Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites.” (The OWASP® Foundation, Inc., 2025) This is what XSS is described as by the OWASP foundation though differently, OWASP doesn’t place XSS in their top 10 like they did with SQL Injection, the MITRE Top 25 Most Dangerous Software Weaknesses list places them as a number 1 threat above SQL Injection (The MITRE Corporation., 2025) This could be explained by Cross-Site Scripting being described by OWASP as being a type of injection with perhaps OWASP placing them in the same category of SQL Injection.

Prevention

Prevention can be difficult as there is no one way to mitigate it as outlined by the massive global network, Cloudflare (no date.) They did however suggest a few ways to help, one of which by setting WAF rules. “A WAF can also be configured to enforce rules which will prevent reflected cross-site scripting. These WAF rules employ strategies that will block strange requests to the server, including cross-site scripting attacks” (Cloudflare, no date.) WAF is short for Web Application Firewall and is described by Cloudflare as something which helps filter and monitor HTTP web traffic as illustrated in this image from Cloudflare:



This is one of the many preventative measures which can be taken to help fight cross-site type attacks, the best option for implementing good WAF Rules is with cloud-based

WAFs as they are easy to implement as well as cheap. Other measures include input validation, sanitising data and adding cookie security measures.

Security Logging & Alerting Failure

Though security logging & alerting failures are only placed at number 9 on the OWASP Top 10 of 2025 (The OWASP® Foundation, Inc., 2025), it is still up there as being important in quite an 'exclusive' way. This vulnerability is essentially how well your system alerts you to any breaches; if you aren't alerted in a timely manner then attacks could easily spiral and expand with other exploits without anyone even noticing before it is too late. As described in the OWASP Top 10: "Without logging and monitoring, attacks and breaches cannot be detected, and without alerting it is very difficult to respond quickly" (The OWASP® Foundation, Inc., 2025) It isn't enough to only try and prevent vulnerabilities but also to detect them as and when they happen.

Prevention

Good alerting comes by ensuring that the correct information is collected and alerted with minimal non-important information. Having too much "fluff" can drown out the important information. According to The OWASP® Foundation, Inc. (2025) there are some important logs to keep such as: failed login attempts, logged in an easy-to-read manner, encoded to prevent log injection/poisoning, every app with security control is logged for failures as well as many more. There is no one simple change which can be implemented to make logs completely airtight, like many vulnerabilities, there is no one solution fits all.

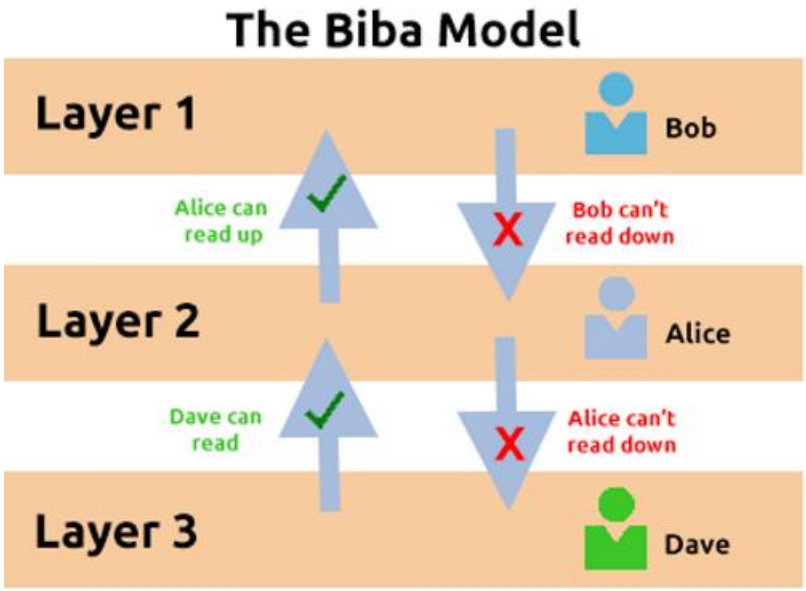
Broken Access Control

The last vulnerability is Broken Access Control. This is where users can perform actions outside of their permitted bounds for their role. OWASP Top 10 has placed this as being the most critical where they found that 100% of the applications they tested had some form of broken access control (The OWASP® Foundation, Inc., 2025). They then go onto describe Broken Access Control as policies wherein users can act outside of their intended role/permissions when they shouldn't be able to. An example, someone could change a URL "user/123" to "user/124" and be able to access another account if not correctly configured. This created GDPR risks as it could lead to data being exposed to parties it shouldn't be.

Prevention

Broken Access Control is a typical example of the principal of least privilege where users should only be given access to what they need access to and nothing more. Some models helping with this concept includes the Biba Model which speaks on only being able to read above your layer and not below allowing data integrity and further explained in the image and further as "This rule means that subjects **can** create or write content to

objects at or below their level but **can only** read the contents of objects above the subject's level (TryHackMe, 2021):



This is further expanded by OWASP when as they state that "Model access controls should enforce record ownership rather than allowing users to create, read, update, or delete any record." (The OWASP® Foundation, Inc., 2025) This essentially copies what the Biba Model does and means individuals own items in their own layer but cannot change anything above that nor can they read or amend below their layer. This is a great way to start implementing good access control, among many other ways.

Keeping in line with GDPR and Data Protection

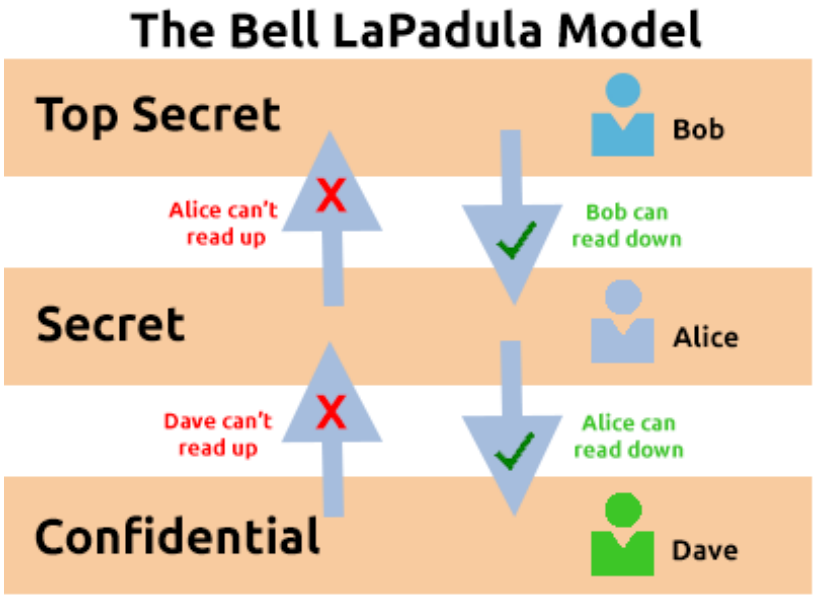
Data Integrity Procedures

Data integrity must be fully upkept when operating a web application database to ensure data is kept correct and true as possible. You should ensure that least-privilege accounts are used as to upkeep the most basic integrity of data, only allowing users access to tables they need. According to leading American multinational technology company IBM corporation (2026), "Your recovery plan should allow for regularly scheduled backup operations". With this, it is important to ensure planned backups are run either incrementally or fully. All inputs should be correctly sanitised, verified or avoided entirely if possible. Audit logs should be always kept. These are a few of many procedures which could be implemented.

GDPR & Cybersecurity Best Practice

GDPR requires organisations to ensure confidentiality, integrity, and availability of personal data as outlined in Article 5(1) (UK GDPR, 2026). The Biba Model is an

essential, but this is not the only model; there is also “The Bell-La Padula Model” (TryHackMe, 2021) ensuring confidentiality:



Data should be encrypted at rest and in transit alongside hashing where required. Backups can help availability of data. Data should also be minimized, only collecting what is required. Most importantly, any breaches must be reported to ICO within 72 hours to ensure fines are not incurred. These practices should be used alongside others as mentioned through this report.

Reference List

Cloudflare (no date) *What is a WAF? | Web Application Firewall explained*. Available from: https://www.cloudflare.com/en-gb/learning/ddos/glossary/web-application-firewall-waf/ [Accessed 17 April 2026].

Cloudflare (no date) *What is cross-site scripting?*. Available from: https://www.cloudflare.com/en-gb/learning/security/threats/cross-site-scripting/ [Accessed 17 April 2026].

UK GDPR [online]. Article 5. legislation.gov.uk. Available from: https://www.legislation.gov.uk/eur/2016/679/article/5 [Accessed 18 April 2026].

IBM Corporation (2026) *Deciding how often to back up*. Available from: https://www.ibm.com/docs/en/db2/12.1.x?topic=strategy-deciding-how-often-back-up [Accessed 18 April 2026].

Mascellino, A. (2026) SQL Injection Flaw Affects 40,000 WordPress Sites. *Infosecurity Magazine* [online]. 03 February. Available from: https://www.infosecurity-magazine.com/news/wordpress-sql-injection-flaw-40000/ [Accessed 29 March 2026].

The MITRE Corporation. (2025) *2025 CWE Top 25 Most Dangerous Software Weaknesses* [online]. MITRE. (no place): The MITRE Corporation. Available from: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html [Accessed 29 March 2026].

The OWASP® Foundation, Inc. (2025) *Broken Access Control* [online]. OWASP. (no place): The OWASP® Foundation, Inc. Available from: https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/ [Accessed 17 April 2026].

The OWASP® Foundation, Inc. (2025) *Cross Site Scripting (XSS)* [online]. OWASP. (no place): The OWASP® Foundation, Inc. Available from: https://owasp.org/www-community/attacks/xss/ [Accessed 17 April 2026].

The OWASP® Foundation, Inc. (2025) *OWASP Top 10:2025* [online]. OWASP. (no place): The OWASP® Foundation, Inc. Available from: https://owasp.org/Top10/2025/ [Accessed 29 March 2026].

The OWASP® Foundation, Inc. (2025) *Security Logging & Alerting Failures* [online]. OWASP. (no place): The OWASP® Foundation, Inc. Available from: https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/ [Accessed 17 April 2026].

TryHackMe (2021) *Principles of Security [online]*. Available from: https://tryhackme.com/room/principlesofsecurity [Accessed 17 April 2026]

PortSwigger (No date) *SQL injection*. Available from: https://portswigger.net/web-security/sql-injection [Accessed 29 March 2026].